

NetID's:

wzw4

nvt5

Known Problems:

Hashtable is about 2-3 times slower than hashmap (java implementation)

Design Decisions:

Search function uses a trie that adds all of the query text's words of query length to search

KeySetClass is a private class, and at first I wanted it to be an array that manually kept track of the keys as a public class, but with that decision removal became a headache, so I decided to make it so that keyset class would just reference hashtable's keys, and iterate over it with hashtable methods.

Hash function is fnv1

Resizing iterates all over the nodes to add it

Testing Strategy:

Coverage seems extensive enough

Tests hash table with other data types

Tests the linked list, and some basic operations explicitly, which the hash table already does

Explicitly tests key functionalities, like key equality and rewriting

Goes through a normal test case with a normal input and sees if all variables are correct

Hash Table uses a mix of black box functional testing, and glass box implementation testing, as well as some randomized tests.

For my hash table, the randomized tests were a bunch of randomly generate length 7 alphabetic strings, which were inserted into the hash table, and then

It explicitly tests the bounds like null hash tables, and the performance graph tested its upper bound since tests won't allow it to get too big.

It also uses glass box testing, as it explicitly tests the resizing logic

Asserts for expected keyset value array

White box testing, tested basic remove, get, and other functions for timings

Spec Revisions:

Any input subclass of Object that should be the key must have a toString() method that doesn't return its memory address (which is reference equality), but obeys state equality.

If you don't override hashCode(), you also don't override equals(), so those two must be overridden in Object such that it has state equality.

Hashing equal objects = equal hashes, then run into hash function that can hash with xor to put it into buckets